

软件工程基础

第 3 章—软件项目管理

Software Project Management

学习笔记 • Study Notes

可行性研究 / 故障树分析 (FTA) / 成本估计 (COCOMO/功能点) / 效益分析

2026 年 6 月 1 日

目录

1 可行性研究概述 —Feasibility Study Overview	3
1.1 什么是可行性研究?	3
1.2 可行性研究的三大内容	3
1.3 可行性研究的步骤	3
1.4 可行性研究可能不可行的原因	3
2 系统组织结构与流程分析 —Organization & Process Modeling	4
2.1 系统组织结构图	4
2.2 系统流程图 —System Flowchart	4
2.3 泳道图 —Swimlane Diagram	4
3 数据流图 (DFD) —Data Flow Diagram	4
3.1 DFD 基本概念	4
3.2 分层 DFD	5
3.3 从 DFD 到物理方案	5
3.4 数据字典 —Data Dictionary	5
4 故障树分析 (FTA) —Fault Tree Analysis	5
4.1 FTA 概述	5
4.2 故障树构建方法	5
4.2.1 顶事件选择	5
4.2.2 边界条件	6
4.2.3 逐层分解	6
4.3 故障树的规范化与简化	6
4.3.1 规范化	6
4.3.2 简化	7
4.4 最小割集定性分析 —Minimal Cut Set (MCS)	7
4.4.1 上行法 (Bottom-Up Method)	7
4.4.2 下行法 (Top-Down Method)	7
4.4.3 布尔代数化简法	7
4.4.4 最小割集分析原则	8
4.5 故障树定量分析	8
4.5.1 定量分析基本假设	8
4.5.2 顶事件概率计算	8
4.5.3 概率近似计算	8
4.5.4 直接法 (不相交化)	9
4.6 重要度分析	9
4.6.1 结构重要度 —Structural Importance	9
4.6.2 概率重要度 —Probability Importance	9
4.6.3 关键重要度 —Critical Importance	9
5 成本估计方法 —Cost Estimation Methods	10
5.1 成本组成	10
5.2 GB/T36964-2018 成本度量规范	10
5.3 估算方法对比	10
5.3.1 自顶向下估计法	11

5.3.2	德尔菲法 (Delphi)	11
5.3.3	由底向上估计法	11
5.4	COCOMO 模型族 — <i>Constructive Cost Model</i>	11
5.4.1	基本层 COCOMO 模型	11
5.4.2	中间层 COCOMO 模型	12
5.4.3	详细层 COCOMO 模型	12
5.5	Putnam 模型	12
5.6	静态单变量模型	12
5.7	功能点方法 (FP) — <i>Function Point</i>	13
5.7.1	五种功能点度量标准	13
5.7.2	GB/T36964 估算过程	13
6	效益分析 — <i>Benefit Analysis</i>	13
6.1	软件项目效益分类	13
6.2	货币的时间价值 — <i>Time Value of Money</i>	14
6.3	纯收入 — <i>Net Income</i>	14
6.4	投资回收期 — <i>Payback Period</i>	14
6.5	投资回收率 — <i>Return on Investment (ROI)</i>	14
6.6	经济效益分析示例	14
	关键公式速查表	16
	本章小结	18

1 可行性研究概述 — *Feasibility Study Overview*

1.1 什么是可行性研究？

核心定义

软件工程项目可行性研究实质是一次大大压缩和简化了的分析和设计过程，主要在较高层次上以较抽象的方式进行。其目的是在尽可能短的时间内以最小的代价确定该项目是否能够开发，是否值得开发。[p.4]

可行性研究不是去研发一个软件项目，而是研究该项目能否在给定的资源和给定的时间开发，是否能够开发，是否值得开发。在尽可能早的时间来评估项目的可行性，不仅可以避免资源的浪费，同时也对加快项目的开发有重要影响。[p.6]

1.2 可行性研究的三大内容

表 1: 可行性研究内容分类

类型	内容说明（中文）	Key Aspects (EN)
技术可行性	待开发系统技术分析、资源有效性分析、风险分析、系统故障分析	Technical feasibility: technology assessment, resource analysis, risk analysis, FTA
经济可行性	成本估计、效益分析	Economic feasibility: cost estimation, benefit analysis
社会可行性	政治意识形态、法律法规、社会道德、民族意识、组织机构或人员	Social feasibility: legal, ethical, organizational, cultural acceptance

[p.7]

1.3 可行性研究的步骤

1. **系统目标和范围的定义** ——要解决的根本问题、达到目标所需的资源和经费；
2. **对现行系统进行分析研究** ——现有系统的物理模型和逻辑模型（入口：现有系统的组织结构）；
3. **导出新系统的逻辑模型** ——解决有关问题后的逻辑模型；
4. **设计新系统的物理方案** ——最先进的方案、实用方案、基本方案；
5. **推荐可行的方案** ——包括推荐理由；
6. **编写可行性研究报告** ——结论：继续、延期和拒绝。

[p.9]

1.4 可行性研究可能不可行的原因

- 没有认真研究市场和竞争对手情况：竞品分析（客观分析、主观分析）
- 数据来源不准确，没有调查记录：实地考察、文献调研、网络搜索
- 数据估算不合理、失真：成本/效益分析方法不符合实际
- 技术方案不合实际：技术不能解决项目目标

[p.5]

2 系统组织结构与流程分析 — *Organization & Process Modeling*

2.1 系统组织结构图

系统的组织结构反映了组织内部的组成以及它们之间的关系。主要包括：

- **组织结构图** (Organization Chart)：每个方框表示一个业务域，通过层次分解精化，直至小的工作组或某个个体。[p.14,15]
- **业务联系图** (Business Relation Diagram)：反映组织各部分与各项业务之间的联系。用 × (辅助组织)、✓ (相关组织)、* (主要业务) 表示关系。[p.16]
- **业务功能树** (Business Function Tree)：一般情况下，组织机构容易发生变化，但组织的业务功能是可以发现的。[p.17]

2.2 系统流程图 — *System Flowchart*

系统流程图是描绘物理系统的传统工具，其基本思想是用图形符号以黑盒子形式描绘系统各部件（如程序、数据库、文档、人工过程等）。系统流程图表达的是信息在系统中各部件之间流动的情况，而不是对信息进行加工处理的控制过程。[p.19,20]

系统流程图建模基本步骤 GB1526-89

1. 建立系统组织结构，并描述组织结构完成哪些业务功能，采用业务联系图或业务功能树描述；
2. 对某个业务处理，考察组织结构的不同业务功能之间的信息流动，采用系统流程图描述；
3. 对建立的系统流程图进行完善。

[p.22,23]

系统流程图的作用：有利于全面了解系统整体业务处理的大概过程；有利于系统分析员、管理员和业务用户相互交流；可用来分析系统业务流程的合理性；是用来构建系统逻辑处理模型的基础。[p.24]

2.3 泳道图 — *Swimlane Diagram*

泳道图（跨职能流程图）显示流程处理中各步骤之间的关系以及执行它们的职能单位。[p.21]

- **水平跨职能流程图**：职能单位位于流程图左侧，泳道水平于画布中摆放——水平排版能够凸显处理流程；
- **垂直跨职能流程图**：职能部门位于流程图上边，泳道垂直于画布摆放——垂直排版更加强调职能单位。

3 数据流图 (DFD) — *Data Flow Diagram*

3.1 DFD 基本概念

将业务功能进一步精化为业务处理，显示每个处理的输入和输出数据对象，指明处理如何变换信息来完成业务功能。

DFD 核心

数据流图描述的是系统的逻辑模型，图中没有具体的物理元素，只是描绘数据在系统中的流动和处理情况。[p.26]

在可行性研究阶段，采用数据流图来描述数据在系统中的流动和处理情况时，不必详细描述，只须概括地描述高层的数据处理和流动。[p.27]

3.2 分层 DFD

- **顶层图 (Context Diagram)**：将整个系统看作一个处理，显示与外部实体的交互；
- **0 层图**：分解顶层处理，显示主要子系统及其间的数据流；
- **1 层图及以下**：对每个处理进一步分解细化。

3.3 从 DFD 到物理方案

一旦建立了一个系统的数据流图，则开发人员可以 and 用户一道检查现存的处理过程，根据系统的目标和提出的有关问题修订数据流图得到新系统的数据流图，即**新系统的逻辑模型**。以此为基础，进一步可给出**新系统的物理方案**，并对每一方案进行可行性研究，最后向用户推荐一种方案。[p.28]

3.4 数据字典 —Data Dictionary

数据字典是对数据流图中所有数据元素的定义的集合，包括：

- 数据流条目：名称、别名、组成、流量等；
- 数据存储条目：名称、编号、组成、组织方式等；
- 处理条目：名称、编号、输入输出、处理逻辑简述等；
- 数据元素条目：名称、类型、长度、取值范围等。

4 故障树分析 (FTA) —Fault Tree Analysis

4.1 FTA 概述

在可行性研究阶段，分析由于软件、硬件、环境和人等各因素造成的系统故障，明确薄弱环节，以便在早期采取相应的措施避免引入故障，确保可信软件开发的可行性。[p.33]

FTA 起源

故障树分析方法由美国贝尔电报公司的电话实验室的沃森 (Waston) 于**1961 年**首先提出。FTA 不需要太多有关系统的具体信息，适合可行性研究阶段。[p.33]

采用故障树分析法进行系统可信性分析是一个**自顶向下**的过程：[p.34]

1. 首先，从要分析的特定故障（或事故）开始，称其为**顶事件**；
2. 接着，建立系统边界条件，明确原因事件和各因素的调查范围；
3. 然后，逐层分析其发生的原因，直到找出故障的基本原因为止，建立故障树。这些基本原因也称为**底事件**或基本事件；
4. 最后，依据故障树进行**定性分析和定量分析**，确定各事件的重要度和事件发生的概率。

4.2 故障树构建方法

4.2.1 顶事件选择

找出后果严重且较易发生的故障作为顶事件：[p.35]

- 熟悉现有系统，调查收集现有系统故障（或事故）案例，进行故障统计；
- 依据新系统需要和用户要求，设想给定新系统可能会发生的故障；

- 对所调查的故障进行全面分析，从中找出后果严重且较易发生的故障作为顶事件；
- 如果系统有多个后果严重且较易发生的故障，则需要将这些故障分别作为顶事件。

4.2.2 边界条件

系统边界条件是指规定所建故障树何处为止： [p.36]

- 不允许出现的事件；
- 不可能发生的事件（小概率事件）；
- 必然事件（在一定条件下必然发生的事件和必然不发生的事件）；
- 与顶事件发生有关的系统单元初始状态和其他假设。

表 2: 边界条件示例

	不能登录	不能出票
顶事件	系统访问失败	出票功能失败
初始状态	系统可响应访问请求	系统有票，用户已支付
不允许出现的事件	外部因素（网络不稳定）	外部因素（网络不稳定）
必然事件	无	系统卖不出票

[p.36]

4.2.3 逐层分解

从顶事件起，找出直接原因的事件，将所有直接原因的事件作为输入事件。依据事件间的逻辑关系，用适当的逻辑门将这些事件关联。如果输入事件还能进一步分解，则将其看作是输出事件（中间事件），进一步找出直接原因的事件。重复上述步骤，逐级分解，直至所有的输入事件不能再分解或不必再分解（底事件）为止。 [p.37,38]

构建原则：

- 明确建立故障树的目标和范围，有助于缩小故障树的规模；
- 顶事件要有明确的定义，避免多义性，以免使故障树逻辑混乱；
- 从顶事件出发，将顶事件的最直接、充分且必要的原因作为顶事件的输入事件；
- 对不同分支中出现的同一共因事件必须使用同一事件符号。

[p.38]

4.3 故障树的规范化与简化

4.3.1 规范化

规范化故障树是指仅有顶事件、中间事件和基本事件三类事件，以及与、或、非三类逻辑门的故障树： [p.41]

- 将未探明事件或当作基本事件或删除；
- 将顺序与门变换为与门；
- 将表决门变换为或门和与门的组合；
- 将禁门变换为与门。

4.3.2 简化

故障树简化是利用**布尔代数**去掉逻辑多余事件和逻辑多余门。可以有效减少故障树的规模，从而减少故障树分析的工作量。注意：故障树不简化不影响故障树定量分析和定性分析的结果。[p.42]

4.4 最小割集定性分析—*Minimal Cut Set (MCS)*

故障树定性分析的目的是求解导致顶事件发生的各种原因以及原因的组合，也就是识别导致顶事件发生的故障模式集合。[p.44]

割集与最小割集

- **割集 (Cut Set)**：故障树中一些基本事件的集合，当这些基本事件同时发生时，顶事件必然发生；
- **最小割集 (MCS)**：若将割集中所包含基本事件中的任意一个去掉就不再是割集了，这样的割集就是最小割集；
- 可靠性关键系统不允许有**单点故障** (Single Point Fault)——系统中单一原因引起整个系统无法运作的故障。

[p.44]

求最小割集的三种方法：[p.45]

4.4.1 上行法 (Bottom-Up Method)

从故障树的基本事件开始，依据故障树的实际结构，逐级向上进行事件的集合运算：[p.45,46]

- 遇到**与门**，将输出事件表示为输入事件的**交**（布尔积）；
- 遇到**或门**，将输出事件表示为输入事件的**并**（布尔和）；
- 遇到**非门**，将输出事件表示为输入事件的反；
- 向上层层代入，并按布尔代数吸收律和幂等律来化简，将顶事件表示为基本事件的若干积项之和的最简式。每一积项对应于故障树的一个最小割集。

上行法示例

基本事件： $X_1, X_2, X_3, X_4, X_5, X_6$

$G_2 = X_3 \cup X_4 \cup X_5$ $G_3 = X_4 \cup X_5 \cup X_6$

$G_1 = G_2 \cap G_3 = (X_3 \cup X_4 \cup X_5) \cap (X_4 \cup X_5 \cup X_6) = (X_3 \cap X_6) \cup X_4 \cup X_5$

$T = X_1 \cup G_1 \cup X_2 = X_1 \cup (X_3 \cap X_6) \cup X_4 \cup X_5 \cup X_2$

全部最小割集： $\{X_1\}, \{X_2\}, \{X_3, X_6\}, \{X_4\}, \{X_5\}$ [p.47]

4.4.2 下行法 (Top-Down Method)

从顶事件开始，自上而下将每个门的输出事件用输入事件替换。遇到与门时，将输入事件并列写在同一行（表示交集）；遇到或门时，将输入事件分别写在不同行（表示并集）。最终得到的每一行即对应一个割集，再通过集合包含关系化简得到最小割集。

4.4.3 布尔代数化简法

直接用布尔代数运算法则（吸收律、幂等律、分配律等）对故障树的结构函数进行化简，化简后的最简与或式中的每一项即为一个最小割集。

4.4.4 最小割集分析原则

阶数：最小割集含基本事件的数目。[p.48]

1. **阶数越小**的最小割集**越重要**；
2. 在**低阶最小割集**中出现的基本事件比高阶最小割集中的基本事件重要；
3. 在最小割集阶数相同的条件下，在不同最小割集中**重复出现次数越多**的基本事件越重要；
4. **最小割集数目越多**，系统的风险性越大。[p.48,49]

4.5 故障树定量分析

在通过故障树定性分析计算得到全部最小割集后，如果能够确定故障树中各个基本事件发生的概率，则可以对故障树进行定量分析。[p.50]

4.5.1 定量分析基本假设

- 基本事件之间**相互独立**；
- 基本事件（顶事件）都只考虑两种状态：发生或不发生（故障或正常）；
- 假定故障分布为**指数分布**；
- 系统是**单调关联系统**：系统所含单元都与系统有关且系统对应的结构函数是单调非减的。[p.51,52]

4.5.2 顶事件概率计算

由于基本事件相互独立，最小割集 C_i 发生的概率等于最小割集中各基本事件发生概率之积：[p.54]

$$P(C_i) = \prod_{X_j \in C_i} P(X_j)$$

若各最小割集相互独立，则顶事件概率：

$$P(T) = \sum_{i=1}^m P(C_i)$$

若各最小割集**不独立**（相交），需采用**全概率法（容斥原理）**：[p.54]

$$P(T) = \sum_{i=1}^m P(C_i) - \sum_{i=1}^{m-1} \sum_{j=i+1}^m P(C_i C_j) + \sum_{i=1}^{m-2} \sum_{j=i+1}^{m-1} \sum_{k=j+1}^m P(C_i C_j C_k) + \cdots + (-1)^{m-1} P(C_1 C_2 \cdots C_m)$$

4.5.3 概率近似计算

当最小割集数 n 特别大时，计算顶事件概率会出现组合爆炸问题。实际工程中常用近似法：[p.55]

一阶近似与二阶近似

一阶近似（上界）： $P_1 = \sum_{i=1}^m P(C_i)$

二阶近似（下界）： $P_1 - P_2$ ，其中 $P_2 = \sum_{i=1}^{m-1} \sum_{j=i+1}^m P(C_i C_j)$

P_1 是 $P(T)$ 的上界， $P_1 - P_2$ 是 $P(T)$ 的下界。

示例计算

设 $P(X_1) = P(X_2) = 0.1, P(X_3) = 0.2, P(X_4) = P(X_5) = 0.3, P(X_6) = 0.1$

最小割集: $\{X_1\}, \{X_2\}, \{X_3, X_6\}, \{X_4\}, \{X_5\}$

一阶近似: $P_1 = 0.1 + 0.1 + 0.2 \times 0.1 + 0.3 + 0.3 = 0.82$

二阶近似: $P_1 - P_2 = 0.82 - 0.236 = 0.584$ [p.56]

4.5.4 直接法（不相交化）

采用集合运算规则，将两个相交集合的并转化成不相交集合的并: [p.57]

$$A \cup B = A \cup (\bar{A} \cap B), \quad P(A \cup B) = P(A) + (1 - P(A))P(B)$$

直接法计算出的顶事件概率比一阶近似的结果小，而比二阶近似的结果大。[p.59]

4.6 重要度分析**4.6.1 结构重要度 — Structural Importance**

基本事件结构重要度分析不考虑基本事件发生的概率，仅从故障树的结构上分析各基本事件的发生对顶事件发生的影响程度。[p.61]

第 i 个基本事件 X_i 的结构重要度系数:

$$I_i^\phi = \frac{1}{2^{n-1}} \sum [\phi(X_1, \dots, X_{i-1}, 1, X_{i+1}, \dots, X_n) - \phi(X_1, \dots, X_{i-1}, 0, X_{i+1}, \dots, X_n)]$$

[p.62]

基于最小割集的近似判断四原则: [p.65]

1. 一阶（单事件）最小割集中的基本事件的结构重要度最大；
2. 仅出现在同一个最小割集中的所有基本事件的结构重要度一样大；
3. 若两个基本事件仅出现在基本事件个数相等的不同最小割集中，则出现次数多的结构重要度大；
4. 若两个基本事件出现在基本事件个数不相等的不同最小割集中，在事件数较少的割集中出现的基本事件结构重要度大。复杂情况按公式近似: $I'_i = \sum_{X_i \in C_j} \frac{1}{2^{n_j-1}}$

[p.65]

4.6.2 概率重要度 — Probability Importance

基本事件概率重要度反映基本事件发生概率变化引起顶事件发生概率变化的程度。[p.67]

$$I_p(X_i) = \frac{\partial F(P(X_1), P(X_2), \dots, P(X_n))}{\partial P(X_i)}$$

其中 F 为顶事件的概率函数， $P(X_i)$ 为事件 X_i 的概率。

4.6.3 关键重要度 — Critical Importance

基本事件 X_i 关键重要度表示基本事件 X_i 概率变化率对顶事件概率变化率的影响程度。[p.69]

$$I_c(X_i) = \frac{P(X_i)}{P(T)} \cdot I_p(X_i)$$

5 成本估计方法 — *Cost Estimation Methods*

5.1 成本组成

一般说来，基于计算机的系统成本主要由四部分组成： [p.74]

- 购置并安装软件/硬件等有关设施的费用；
- 系统开发费用；
- 系统安装、运行和维护费用；
- 人员培训费用等。

5.2 GB/T36964-2018 成本度量规范

GB/T36964-2018《软件工程-软件开发成本度量规范》认为，软件开发成本包括软件开发过程中的所有人力成本和非人力成本之和。 [p.75]

表 3: GB/T36964-2018 成本分类

成本类别	说明
直接人力成本	开发单位项目组成员的工资、奖金和福利等人力资源费用。非全职人员按其项目中投入的工作量占项目总工作量的比例折算。
间接人力成本	开发单位服务于开发管理整体需求的非项目组人员的人力资源费用分摊（如部门经理、PMO 人员）。
直接非人力成本	办公费、差旅费等。
间接非人力成本	不为某个特定项目但服务于整体开发活动的非人力成本（房租、水电、知识产权等）。

[p.75]

5.3 估算方法对比

在可行性研究阶段，估算项目成本及工作量通常可以采用以下几种方法： [p.76]

表 4: 成本估算方法对比

方法	优点	缺点	适用场景
自顶向下	工作量小、速度快	易低估局部问题，无借鉴时偏差大	有类似项目经验
德尔菲法 (Delphi)	匿名性、反馈性、统计性，结果可靠	耗时较长（3-5 轮）	估计人数较多
由底向上	各任务由开发人员估计，较准确	全局花费（如质量管理）易被忽视	需求明确，可细化
经验模型 (CO-COMO 等)	客观、结果可重复	需合理校准，必须慎重使用	有历史数据支撑

5.3.1 自顶向下估计法

依据先前已完成项目所耗费的总成本（总工作量），推算新开发软件的总成本，然后在项目内部进行成本分配。[p.78]

5.3.2 德尔菲法 (Delphi)

1. 组织者将系统定义文件或规格说明发放给每位估计人员；
2. 估计人员各自单独进行成本估算，并填入有关表格，说明有关理由；
3. 组织者综合估计人员的估计，并对个别与众不同的估计进行单独讨论，将估计结果分类摘要通知大家；
4. 重复上述估计过程，直到大家的意见接近为止（一般 3-5 轮）。

特点：匿名性、反馈性、统计性[p.79]

5.3.3 由底向上估计法

- 软件范围包括功能、性能、限制、接口和可靠性等；
- 对软件范围进行适当的细化，以提供较详细的细节；
- 将各任务单元的成本汇合成项目的总共工作量和成本。
- 示例：CAD 应用开发软件包—细化为用户界面及控制、二维几何分析、三维几何分析、数据库管理、计算机图形显示、外设控制和设计分析模块等 7 个主要功能。[p.82-85]

5.4 COCOMO 模型族 — *Constructive Cost Model*

玻姆 (Boehm) 将软件成本估算分成 3 个由粗到细的层次：基本层、中间层和详细层。每个层次又按软件项目的应用领域和复杂程度分成 3 种类型。[p.90]

$$MM = a \times (KDSI)^b \times \prod_{i=1}^{15} f_i$$

$$TDEV = c \times (MM)^d$$

其中：MM—开发工作量（人月），KDSI—源指令条数（千行），TDEV—开发时间（月）， f_i —15 项项目影响调节因子， a, c —模型系数， b, d —模型指数。[p.90]

5.4.1 基本层 COCOMO 模型

不考虑成本影响调节因子，是对软件成本的一种宏观粗略估计，是一个静态单变量模型。[p.91]

表 5: 基本层 COCOMO 模型参数

类型	工作量 MM	开发时间 TDEV	适用范围
组织型 (Organic)	$2.4 \times (KDSI)^{1.05}$	$2.5 \times (MM)^{0.38}$	较小、较简单的软件项目
半独立型 (Semi-detached)	$3.0 \times (KDSI)^{1.12}$	$2.5 \times (MM)^{0.35}$	介于组织和嵌入之间
嵌入型 (Embedded)	$3.6 \times (KDSI)^{1.20}$	$2.5 \times (MM)^{0.32}$	严格约束下开发的软件

[p.91]

5.4.2 中间层 COCOMO 模型

主要考虑了从整个生存期来衡量成本影响调节因子，共**15 项**，分成 4 类：[p.92,93]

- **产品属性**：软件可靠性、数据库规模、产品复杂度；
- **硬件属性**：执行时间约束、主存约束、虚拟机的易变性、环境周转时间；
- **人员属性**：分析员能力、应用领域经验、程序员能力、虚拟机使用经验、编程语言经验；
- **项目属性**：现代编程实践、软件工具使用、开发进度约束。

5.4.3 详细层 COCOMO 模型

需要考虑各调节因子对于不同开发阶段的影响。针对每一个影响因素，按模块级、子系统级和系统级，有三张工作量因素分级表，供不同级别的估算使用。[p.95]

COCOMO II：玻姆等于 2000 年提出，可以处理较广泛的软件工程技术：面向对象、包含瀑布模型及其他生存期模型、复用、第四代语言等。[p.96]

COCOMO 评价

玻姆的自我评价：一个软件估算模型能够在成本估算上相差不到 20%，时间估算上相差不到 70%，就相当不错了。当开发产品时，实际的开发工作量应与预测值进行比较，出现偏差可作为一个有问题的早期警告。[p.96]

5.5 Putnam 模型

普特南（Putnam）依据大型软件工程项目（总工作量达到或超过 30 年）的数据，提出 Putnam 模型，适用于规模在 100KLOC 以上的软件。[p.89]

Putnam 方程式

$$L = C_k \times K^{1/3} \times t_d^{4/3}$$

其中： L —源代码行数（LOC）， K —软件全生存周期（含维护）所需工作量（人年）， t_d —项目开发持续时间（年）， C_k —技术状态常数。

反解：

$$K = \frac{L^3}{C_k^3 \times t_d^4}, \quad t_d = \left[\frac{L^3}{C_k^3 \times K} \right]^{1/4}$$

这表明：**如果缩短开发时间，将意味着增加项目的开发工作量。**

[p.89]

5.6 静态单变量模型

典型结构为： $E = A + B \times (\text{估计变量})^C$ ，其中 A 、 B 和 C 是由经验导出的常数； E 是以人月（PM）为单位的工作量。[p.88]

例如 Walston 和 Felix 模型：

$$\begin{aligned} E &= 5.2 \times (KLOC)^{0.91} & D &= 4.1 \times (KLOC)^{0.36} = 13.47 \times E^{0.35} \\ DOC &= 49 \times (KLOC)^{1.01} & S &= 0.54 \times E^{0.6} \end{aligned}$$

其中 KLOC 表示千行源代码行数。[p.88]

5.7 功能点方法 (FP) —*Function Point*

软件规模估算常用的国际标准方法。

5.7.1 五种功能点度量标准

- SJ/T11617—2016 COSMIC-FFP 功能规模测量方法
- SJ/T11618—2016 MK II 功能点分析计数实践指南
- SJ/T11619—2016 功能规模测量 NESMA 方法
- SJ/T11620—2016 FiSMA1.1 功能规模测量方法
- ISO/IEC20926:2009 IFPUG 功能规模测量方法

[p.101]

其中,IFPUG和NESMA是目前最主要的标准——国际基准比对组织中超过 90% 的数据采用 IFPUG/NESMA 方法,国内行业数据 100% 采用 IFPUG/NESMA 方法。[p.101]

在适用 IFPUG 或 NESMA 方法时,可以根据需求的粒度和管理需要,选择**预估功能点方法**、**估算功能点方法**或者**详细功能点方法**。[p.100]

5.7.2 GB/T36964 估算过程

四个估算步骤,层层递进,逐步细化: [p.99]

1. 规模估算 (采用功能点方法);
2. 工作量估算 (根据行业基准数据获取各类工作的工作量占比);
3. 工期估算;
4. 成本估算 (直接人力成本 + 间接人力成本 & 非人力成本分摊)。

注意事项

需求模糊或不确定时,采用类比或类推法估算工作量或成本。若当前的项目需求极其模糊或不确定,可不进行规模估算,而直接采用类比法或类推法估算工作量和成本。[p.99,100]

6 效益分析 —*Benefit Analysis*

6.1 软件项目效益分类

软件项目效益有两部分: [p.109,110]

表 6: 经济效益与社会效益对比

效益类型	具体表现 (中文)	Examples (EN)
经济效益 (Economic Benefits) 有形的	提高生产效率、降低生产成本、提升产品质量、缩短生产周期、降低物资库存	Efficiency, cost reduction, quality improvement, shorter cycles
社会效益 (Social Benefits) 无形的	推动技术创新、提升员工技能、改善工作环境、推动社会发展、加强单位管理	Innovation, skill development, work environment, social impact

6.2 货币的时间价值 — *Time Value of Money*

通常投资在前，取得效益在后，因此要考虑货币的时间价值。货币的时间价值常用**利率**的形式来表示。

[p.112]

货币时间价值公式

终值 (Future Value): $F = P \times (1 + i)^n$

现值 (Present Value) / 贴现: $P = \frac{F}{(1 + i)^n}$

其中: P —现值, F —终值, i —计息期利率, n —计息期数。

$(1 + i)^n$ 被称为**复利终值系数**, 用符号 $(F/P, i, n)$ 表示。

[p.112]

6.3 纯收入 — *Net Income*

纯收入值等于整个生存周期内系统的累积经济效益（折算成当前值）与投资之差。[p.113]

这相当于比较投资开发一个软件系统和把钱存在银行中（或贷给其他企业）这两种方案的优劣。**只有当纯收入大于零时才能考虑投资。**

6.4 投资回收期 — *Payback Period*

投资回收期的值等于使累计的经济效益（折算成当前值）等于最初投资所需要的时间。[p.114]

投资回收期越短，就能越快获得经济效益，因而这项工程也就越值得投资。

6.5 投资回收期 — *Return on Investment (ROI)*

投资回收期可用来衡量投资效益的大小，并可以和银行年利率进行比较。[p.115]

$$P = \frac{F_1}{(1 + j)} + \frac{F_2}{(1 + j)^2} + \cdots + \frac{F_n}{(1 + j)^n}$$

其中: P —当前投资额, F_i —第 i 年年底的效益 ($i = 1, 2, \dots, n$), n —系统使用寿命, j —投资回收期。

6.6 经济效益分析示例

示例

某项目开发的系统取代现有系统，可每年节省 40 万元。项目开发投资 110 万。预计系统寿命 5 年，银行利率 0.03。

- 第 1 年收回: $40/1.03 = 38.8$ 万; 第 2 年: $40/1.03^2 = 37.7$ 万;
- 第 3 年收回: $40/1.03^3 = 36.6$ 万; 第 4 年: $40/1.03^4 = 35.5$ 万;
- 第 5 年收回: $40/1.03^5 = 34.5$ 万;
- **累计效益 (现值):** 183.1 万;
- **纯收入:** $183.1 - 110 = 73.1$ 万;
- **投资回收期:** $2 + (110 - 38.8 - 37.7)/36.6 = 2.91$ 年;
- **投资回收期:** 在 0.23 到 0.24 之间。

[p.116]

管理者根据有关分析结果，结合其它潜在的对投资的使用，考虑有关社会效益，可以在经济上确定系统是否值得投资开发。[p.117]

关键公式速查表 — *Key Formula Reference*

表 7: FTA 定量分析公式汇总

项目	公式
最小割集概率	$P(C_i) = \prod_{X_j \in C_i} P(X_j)$
顶事件概率 (全概)	$P(T) = \sum P(C_i) - \sum P(C_i C_j) + \sum P(C_i C_j C_k) - \dots$
一阶近似 (上界)	$P_1 = \sum P(C_i)$
二阶近似 (下界)	$P_1 - P_2$, 其中 $P_2 = \sum \sum P(C_i C_j)$
结构重要度	$I_i^\phi = \frac{1}{2^n - 1} \sum [\phi(\dots, 1, \dots) - \phi(\dots, 0, \dots)]$
概率重要度	$I_p(X_i) = \frac{\partial F}{\partial P(X_i)}$
关键重要度	$I_c(X_i) = \frac{P(X_i)}{P(T)} \cdot I_p(X_i)$

表 8: 成本与效益公式汇总

项目	公式
静态单变量	$E = A + B \times (ev)^C$
Walston-Felix	$E = 5.2 \times (KLOC)^{0.91}$
COCOMO	$MM = a \times (KDSI)^b \times \prod f_i, TDEV = c \times (MM)^d$
Putnam	$L = C_k \times K^{1/3} \times t_d^{4/3}$
终值	$F = P(1 + i)^n$
现值	$P = F / (1 + i)^n$
投资回收期	$P = \sum_{k=1}^n \frac{F_k}{(1+j)^k}$

方法对比总表 —*Method Comparison Matrix*

表 9: 可行性研究各分析方法对比

方法	核心理念	主要输出	适用阶段
组织结构图	描述组织内部组成与关系	组织结构图、业务功能树	前期调研
系统流程图	黑盒子描绘物理系统	物理处理流程	业务流程分析
泳道图	强调执行职能部门	跨职能流程	业务流程分析
DFD 数据流图	数据流动的逻辑模型	分层数据流图、数据字典	数据流分析
FTA 故障树	自顶向下分析故障原因	故障树、最小割集、概率	可信性分析
自顶向下估计	类比已完成项目	总成本/总工作量	粗略估算
Delphi 法	多轮匿名估计	收敛的成本估计	多人参与
由底向上估计	任务分解汇总	详细成本估计	需求明确
COCOMO	经验参数模型	MM/TDEV	三种层次
Putnam	动态多变量	K/td	大型项目 (100KLOC+)
功能点 (FP)	功能规模度量	功能点数	规模估算

本章小结 —Chapter Summary

本章系统介绍了软件项目可行性研究的完整体系：

1. **可行性研究概述**：技术可行性、经济可行性、社会可行性三大方面，以及可行性研究的六个步骤。
2. **系统建模方法**：组织结构图、系统流程图、泳道图、DFD 数据流图，构成从物理到逻辑的建模体系。
3. **故障树分析 (FTA)**：自顶向下分析系统故障原因的有力工具——顶事件选择、故障树构建与规范简化、最小割集定性分析（上行法/下行法/布尔代数法）、概率定量分析（全概法/近似法/直接法）、三大重要度分析（结构/概率/关键）。
4. **成本估计**：自顶向下、Delphi 法、由底向上三大估计方法的对比；COCOMO 三层模型族（基本/中间/详细）与 COCOMO II；Putnam 动态多变量模型；GB/T36964-2018 行业标准；功能点方法（IFPUG/NESMA）。
5. **效益分析**：经济效益与社会效益；货币时间价值（终值/现值/贴现）；纯收入、投资回收期、投资回收率的计算与工程判断准则。

——第3章软件项目管理学习笔记完——